

Categories, Proofs, and Processes:

Lecture 11 Naturality

Wira Azmoon Ahmad

8 Feb 2024

4 Naturality

Functors treat categories as objects. We can treat functors themselves as objects, so what are the arrows?

4.1 Motivating Examples

Suppose a category $\mathcal{C}$ has products, we have two isomorphic ternary products.

$$(A \times B) \times C \cong A \times (B \times C)$$

Note that this isomorphism does not really depend on the specific objects and arrows involved, i.e. the following commutes

4.1.1 Associators

$$\begin{array}{ccc} (A \times B) \times C & \longrightarrow & (A' \times B) \times C \\ i \downarrow & & \downarrow i' \\ A \times (B \times C) & \longrightarrow & A' \times (B \times C) \end{array}$$

More generally we have

$$\begin{aligned} (- \times -) \times - &: \mathcal{C} \times \mathcal{C} \times \mathcal{C} \rightarrow \mathcal{C} \\ - \times (- \times -) &: \mathcal{C} \times \mathcal{C} \times \mathcal{C} \rightarrow \mathcal{C} \end{aligned}$$

4.1.2 Lists

Consider the list constructor

$$x_0 : [x_1, x_2, \dots, x_n] = [x_0, x_1, x_2, \dots, x_n]$$

”Liste” constructor in reverse

$$[x_n, x_{n-1}, \dots, x_1] : x_0 = [x_n, x_{n-1}, \dots, x_1, x_0]$$

Consider the functor $\text{List} : \mathbf{Set} \rightarrow \mathbf{Set}$, which maps a set X to $\text{List}(X)$, the set of all finite lists with elements in X .

$$\begin{array}{ccc} \text{List}(X) & \xrightarrow{\text{List}(f)} & \text{List}(Y) \\ \text{reverse} \downarrow & & \downarrow \text{reverse} \\ \text{List}(X) & \xrightarrow{\text{List}(f)} & \text{List}(Y) \end{array}$$

4.1.3 More Examples

Consider functors $G : \underline{2}_{\Rightarrow} \rightarrow \mathbf{Set}$.

$$\underline{2}_{\Rightarrow} = \bullet \begin{array}{c} \xrightarrow{\quad} \\ \xleftarrow{\quad} \end{array} \bullet$$

corresponds to directed graphs

$$E \begin{array}{c} \xrightarrow{s} \\ \xleftarrow{t} \end{array} V$$

where $E \subseteq V \times V$.

$$\begin{array}{ccc} \text{Graph 1} & E & \xrightarrow{s} V \\ & \downarrow h & \downarrow h \\ \text{Graph 2} & E' & \xrightarrow{s'} V' \end{array}$$

is a graph homomorphism.

4.1.4 Unnatural transformation

$$\text{reverse}_{\text{Bool}} : \text{List}(X) \rightarrow \text{List}(X) := \begin{cases} \text{reverse list} & \text{if } X = \text{Bool} \\ \text{id} & \text{otherwise} \end{cases}$$

Consider the function $\text{odd} : \mathbb{N} \rightarrow \text{Bool}$ (True if $n \in \mathbb{N}$ is odd, False otherwise).

$$\text{reverse}_{\text{Bool}} \circ \text{list odd}([1, 3, 2]) = [F, T, T]$$

$$\text{list odd} \circ \text{reverse}_{\text{Bool}}([1, 3, 2]) = [T, T, F]$$

Because the function cares about the types inside, it's unnatural. The diagram will not commute.

4.2 Natural Transformations

Definition 4.1. Let $F, G : \mathcal{C} \rightarrow \mathcal{D}$ be functors. A natural transformation

$$t : F \rightarrow G$$

is a family of morphisms in $\mathcal{D}$ indexed by objects A of $\mathcal{C}$,

$$\{t_A : F A \rightarrow G A\}_{A \in \text{Ob}(\mathcal{C})}$$

such that, for all $f : A \rightarrow B$, the following diagram commutes

$$\begin{array}{ccc} F A & \xrightarrow{F f} & F B \\ t_A \downarrow & & \downarrow t_B \\ G A & \xrightarrow{G f} & G B \end{array}$$

This condition is called naturality.

Definition 4.2. If each t_A is an isomorphism, we say t is a natural isomorphism:

$$t : F \cong G$$

Note 4.3. A natural isomorphism is precisely an isomorphism in the functor category

$$\text{Func}(\mathcal{C}, \mathcal{D}) := \mathcal{D}^{\mathcal{C}}$$

4.6 Equivalence of categories

Definition 4.4 (Def. 57 (pg. 38)). An equivalence of categories consists of a pair of functors

$$E : \mathcal{C} \rightarrow \mathcal{D}$$

$$F : \mathcal{D} \rightarrow \mathcal{C}$$

and a pair of natural isomorphisms

$$\alpha : 1_{\mathcal{C}} \cong F \circ E$$

$$\beta : 1_{\mathcal{D}} \cong E \circ F$$

F is the pseudo-inverse of E . $\mathcal{C}, \mathcal{D}$ are then said to be equivalent, denoted $\mathcal{C} \simeq \mathcal{D}$.

Note 4.5. This is an alternative definition to a functor being an equivalence.

Proposition 4.6 (Ex. 1.4.4(2)(b) $\Rightarrow$ (a)). *If functor $F : \mathcal{C} \rightarrow \mathcal{D}$ is (part of) an equivalence of categories, it is an equivalence - full, faithful, and*

$$\forall D \in \text{Ob}(\mathcal{D}), \exists C \in \text{Ob}(\mathcal{C}). F C \cong D$$

Proof. For faithfulness, take $E : \mathcal{D} \rightarrow \mathcal{C}$ such that

$$\alpha : 1_{\mathcal{C}} \cong EF$$

$$\beta : 1_{\mathcal{D}} \cong FE$$

are natural transformations in $\mathcal{C}$, for any object C , we have $\alpha_C : C \cong EF(C)$ and

$$\begin{array}{ccc} C & \xrightarrow{f} & C' \\ \alpha_C \downarrow & & \downarrow \alpha_{C'} \\ EF(C) & \xrightarrow{EF(f)} & EF(C') \end{array}$$

Thus if $F(f) = F(f')$ then $EF(f) = EF(f')$ and so $f = f'$. So F is faithful since

$$F_{C,C'} : \mathcal{C}(C, C') \rightarrow \mathcal{D}(F(C), F(C'))$$

is surjective.

For fullness, consider any arrow $h : F(C) \rightarrow F(C')$ in $\mathcal{D}$. We note that $C \cong EF(C)$ and $C' \cong EF(C')$. So consider $E(h)$ then there is an

$$f = (\alpha_{C'})^{-1} \circ E(h) \circ \alpha_C : C \rightarrow C'$$

where $\alpha_C, \alpha_{C'}$ are iso. By naturality we have $F(f) : F(C) \rightarrow F(C')$ that

$$\begin{array}{ccc} C & \xrightarrow{f} & C' \\ \alpha_C \downarrow & & \downarrow \alpha_{C'} \\ EF(C) & \xrightarrow{EF(f)} & EF(C') \end{array}$$

So $EF(f) = E(h) : EF(C) \rightarrow EF(C')$. Since E is faithful, $EF(f) = E(h) \implies F(f) = h$, so F is full.

For essential surjectivity on objects for any $D \in \text{Ob}(\mathcal{D})$, we have

$$t : 1_{\mathcal{D}} \cong FE$$

so for any $D \in \text{Ob}(\mathcal{D})$,

$$\beta_D : D \cong FED = F(ED)$$

for $ED \in \text{Ob}(\mathcal{C})$. □

4.7 Representable functors

Definition 4.7. A functor $F : \mathcal{C} \rightarrow \mathbf{Set}$ is said to be representable if it is naturally isomorphic to the covariant Hom-functor at A , $\mathcal{C}(A, -) : \mathcal{C} \rightarrow \mathbf{Set}$ for some A .

Example 4.8. Forgetful functor $U : \mathbf{Mon} \rightarrow \mathbf{Set}$. We want that there is a monoid Z such that for all M ,

$$\mathbf{Mon}(Z, M) \cong U(M), \quad Z(\mathbb{N}, +, 0)$$